

Chapter 05: Mid-Interview Curveball & Requirement Evolution Mastery

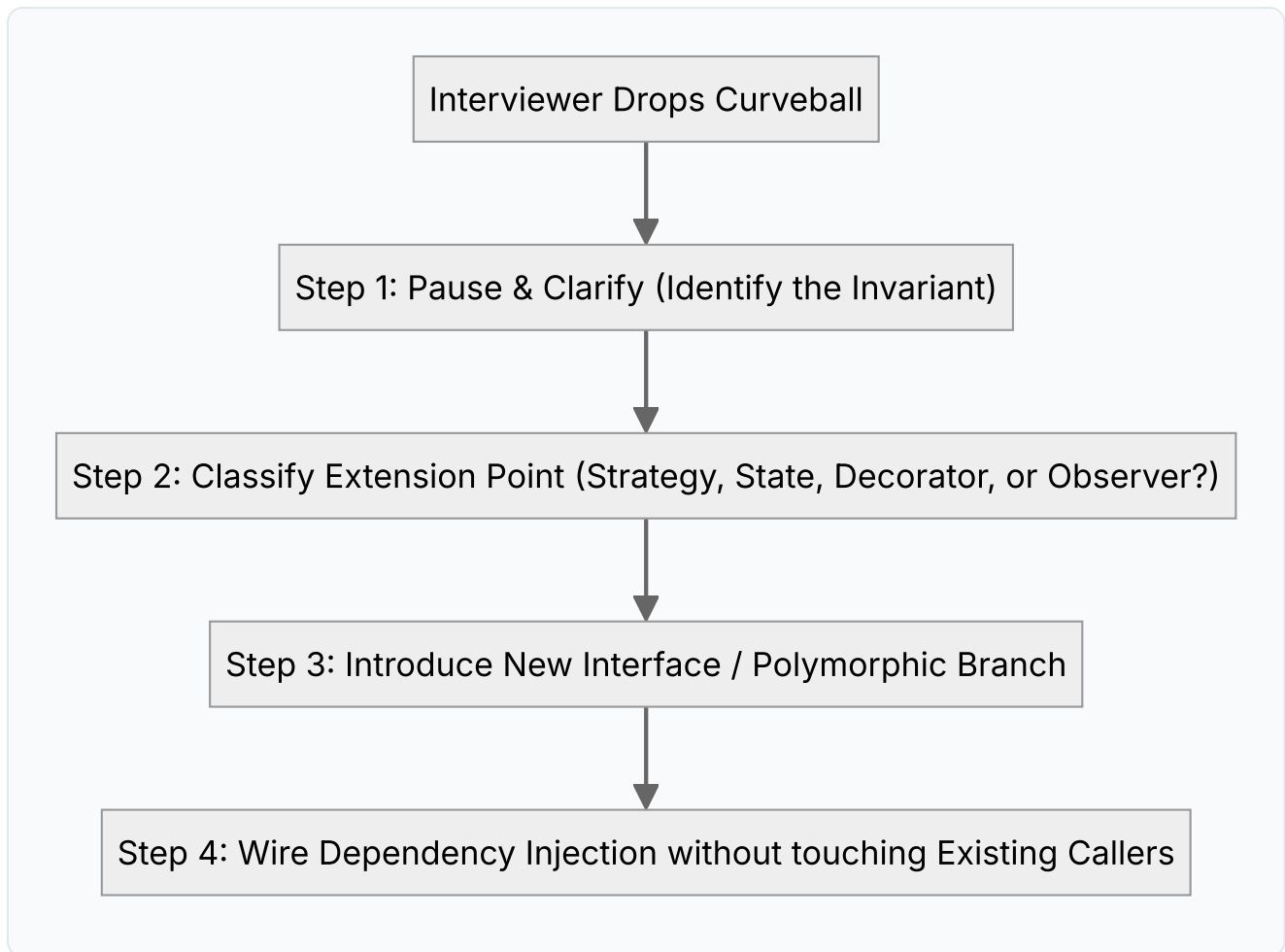
The 25-Minute Trap In FAANG/top-tier LLD rounds, the interviewer rarely lets you finish your initial clean design unchallenged. Around the 25-minute mark, just as your class diagrams look tidy and your interfaces are written, the interviewer will lean back and say:

"What if demand suddenly surges during rainstorms and we need dynamic surge pricing?" or "What if seats need a 10-minute temporary lock that automatically expires and releases inventory if payment fails?"

If you freeze, rewrite your entire class model, or patch the code with dirty `if-else` flags, you fail the test. The curveball is not meant to see if you can code faster—it is a deliberate stress test to see if your architecture was truly extensible (Open/Closed Principle).

1. The 4-Step Curveball Response Framework

When a curveball strikes, never touch your keyboard immediately. Follow this 4-step protocol:



1. Step 1: Clarify Invariants & Scope

- *Verbalize*: "Great requirement. Let me clarify: is this a global rule change, or must the system support both legacy behavior and the new behavior concurrently based on configuration?"

2. Step 2: Isolate the Volatility

- Ask yourself: Is what changed an **algorithm** (Strategy), a **lifecycle phase** (State), a **notification reaction** (Observer), or an **additive feature** (Decorator)?

3. Step 3: Modify Interfaces, Don't Hack Concrete Classes

- Never add boolean flags (`is_surge_pricing_enabled=True`). Instead, extract the existing behavior into a default implementation of an abstract strategy.

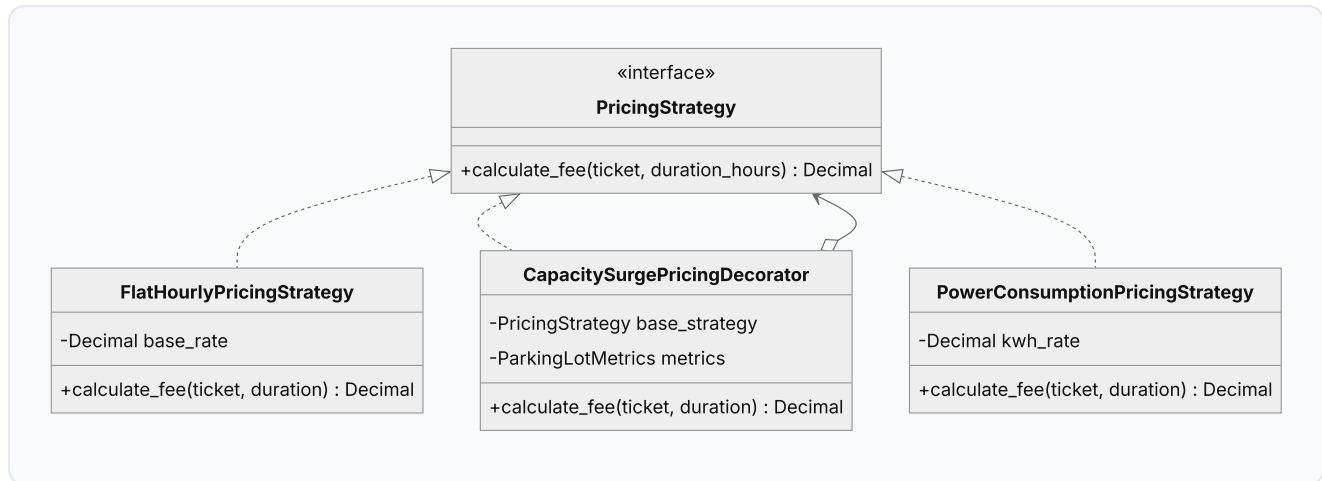
4. Step 4: Prove Zero Regression

- Show that existing unit tests continue to pass without modification.

2. Curveball 1: Parking Lot → Dynamic Surge Pricing & EV Charging Queues

The Scenario

You designed a standard Parking Lot with Hourly billing (Compact , Large , Handicapped).
Interviewer: "Our parking lot is now at an airport. When capacity exceeds 80%, rates double. Furthermore, Tesla drivers can request EV fast charging, which charges per kWh instead of per hour and needs its own queue."

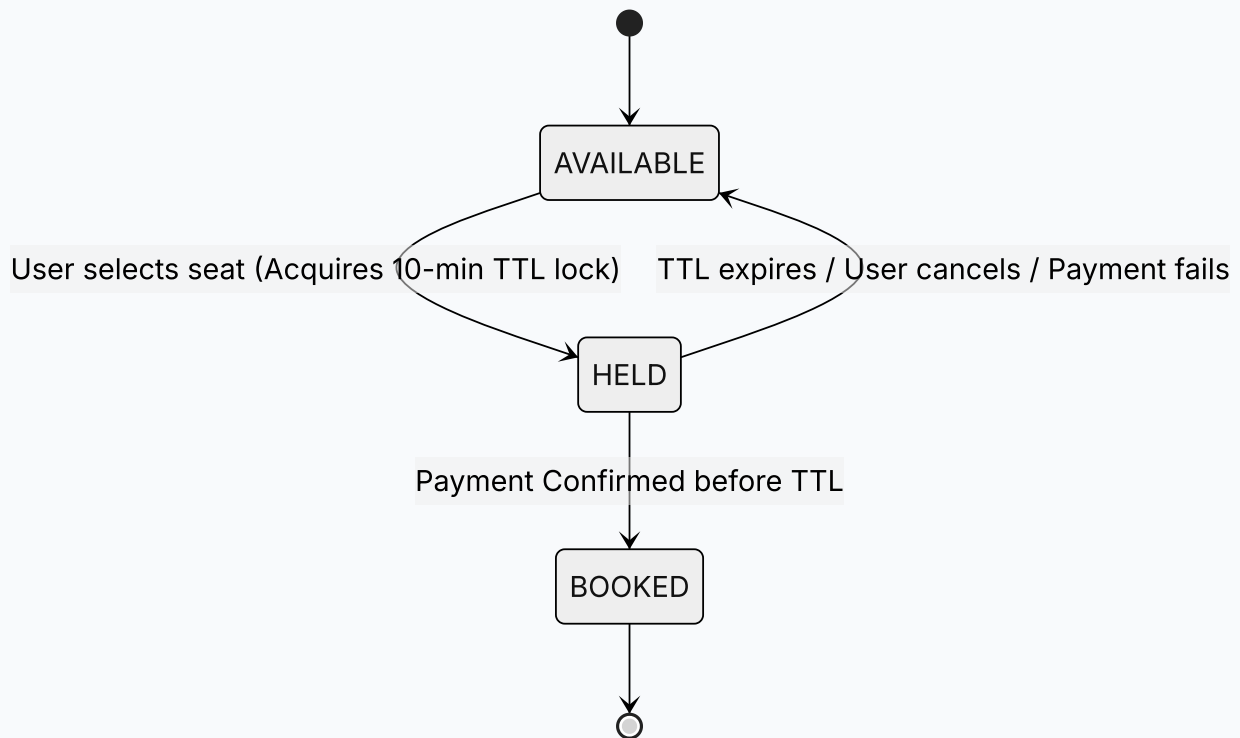


The Code Transformation

3. Curveball 2: Movie Booking → 10-Minute Temporary Seat Hold with Expiry

The Scenario

You designed a Movie Booking system where seats are reserved and paid immediately. *Interviewer: "Users are colliding on seat selection. When a user selects a seat, hold it for 10 minutes. If they don't complete payment within 10 minutes, release it automatically so other users can see it."*



The Architectural Evolution: State Pattern with Ephemeral Leases

4. Curveball 3: Elevator System → Firefighter Emergency Mode

The Scenario

You designed an elevator dispatch algorithm (SCAN / LOOK). *Interviewer: "A fire alarm sounds. We need all elevators to immediately cancel all passenger calls, disable hall buttons, drop to Floor 1 at maximum speed, open doors, and lock until a firefighter key turns."*

The Architectural Evolution: State Pattern with Priority Interrupt

5. Summary Matrix: How to Defeat Any Curveball

Curveball Signal	What Changed?	Architectural Tool
<i>"Calculate charges differently for X"</i>	Algorithm / Computation	Strategy Pattern or Decorator Pattern
<i>"When X happens, suddenly notify Y and Z"</i>	Reaction / Side-Effect	Observer Pattern (Publish/Subscribe)
<i>"System behaves completely differently in mode X"</i>	Internal State / Lifecycle	State Pattern
<i>"Add audit logging / rate limiting to existing API"</i>	Cross-cutting concern	Proxy or Decorator Pattern
<i>"Support a new third-party vendor with weird API"</i>	Protocol / Schema mismatch	Adapter Pattern

Further Reading

- [Wikipedia: Open-closed principle](#)
- [Refactoring Guru: Strategy](#)
- [Refactoring catalog](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▼ 1. A new requirement arrives mid-interview. What is the first thing you do?

Restate it, identify which abstraction it affects, and show the minimal change: ideally adding a class, not editing many.

- ▼ 2. How do you show your design is extensible?

Point at the interface where the change plugs in and demonstrate it with a concrete new implementation.

- ▼ 3. What is a sign that your abstraction is wrong?

A new requirement forces edits across many unrelated classes or conditionals on type.

- ▼ 4. Why avoid over-engineering for hypothetical futures?

Unneeded flexibility adds complexity now; design for stated requirements and keep seams for likely change.